

Отладка шаблона контроля Wasaby

Структура скомпилированного шаблона

В процессе сборки дистрибутива или локального проекта все шаблоны wml и tmpl компилируются в JavaScript-модули. При инициализации модуля, первом обращении к модулю с помощью функции **require**, выполняется построение шаблонной функции шаблона по его мета-описанию.

Мета-описание содержит информацию, непосредственно относящуюся к шаблону: корневые узлы шаблона, контентные опции контролов и шаблонов, определения inline-шаблонов, коллекцию имен реактивных свойств, коллекцию Mustache выражений и некоторые другие служебные данные.

Скомпилированный шаблон имеет следующий общий вид, debug-версии которого для удобства отладки сопровождаются дополнительными комментариями:

▼ Пример кода

```
1  define("wml!Module/template", ["Compiler/IR"], function(IR) {
2      "use strict";
3
4      return IR.generate({
5          v: /* Версия компилятора */ 1,
6          m: /* Имя текущей зависимости */ "wml!Module/template"
7          d: /* Коллекция внешних зависимостей, если есть */ [],
8          t: /* Коллекция шаблонов */ [
9              /* 0 */ IR.wrapTemplateBody("templateName", () => [
10                 /* тело шаблона ws:template */
11             ]),
12             /* 1 */ IR.wrapContentBody(() => [
13                 /* тело контентной опции */
14             ], "content"),
15             /* 2 */ IR.wrapRootBody(() => [
16                 /* тело корневого шаблона */
17             ]),
18         ],
19         p: /* Коллекция имен реактивных свойств шаблона, если есть */
20         [],
21         e: /* Коллекция Mustache-выражений, если есть */ [
22             /* 0 */ (methods, data) => (/* тело выражения */),
23             /* 1 */ (methods, data) => (/* тело выражения */),
24             /* 2 */ (methods, data) => (/* тело выражения */),
25         ]
26     });
27 });
```

В местах использования inline-шаблонов, которые определяются с помощью ws:template, используются индексы из таблицы шаблонных функций с комментарием, содержащим имя inline-шаблона:

▼ Пример кода

```
generator.createInline(context, /* baseLookupInputTemplate */ 3, {/*
конфигурация */})
```

В местах использования контентных функций также используются индексы из таблицы шаблонных функций:

▼ Пример кода

```
1  {
```

```

2     ...
3     content: generator.createContentOption(context, 4),
4     ...
5 }

```

В местах вычисления mustache-выражений используются индексы из таблицы mustache-выражений, которые дополняются комментарием, который содержит текст оригинального выражения, например:

▼ Пример кода

```
generator.evalExpression(context, /* model.getValue("option") */ 5)
```

В местах вычисления циклов также используются индексы из таблицы Mustache-выражений, например, для следующих циклов в стилях `foreach` и `for`

▼ Пример кода

```

1 <ws:for data="index, item in collection">
2     <!-- тело цикла -->
3 </ws:for>
4
5 <ws:for data="iterator.init(); iterator.test(); iterator.update()">
6     <!-- тело цикла -->
7 </ws:for>

```

соответствуют следующие фрагменты из скомпилированного шаблона.

▼ Пример кода

```

1 generator.foreach(
2     context, 0,
3     ["item", "index"],
4     /* collection */ 6,
5     (generator, context) => ([/* тело цикла */])
6 )
7
8 generator.for(
9     context, 1,
10    /* iterator.init() */ 7,
11    /* iterator.test() */ 8,
12    /* iterator.update() */ 9,
13    (generator, context) => ([/* тело цикла */])
14 )

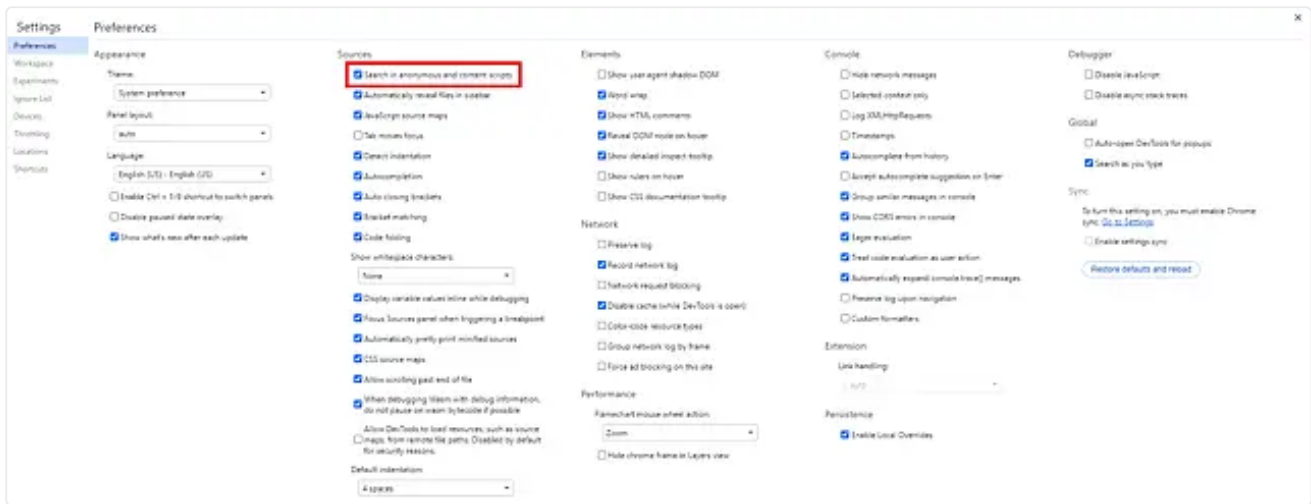
```

Перед отладкой

Search in anonymous and content scripts

При отладке шаблонов, скомпилированных на клиенте, требуется выполнять поиск внутри анонимных скриптов, и чтобы это стало возможным, необходимо включить поддержку данного функционала.

В браузере Google Chrome это выполняется со страницы инструментов разработчика Chrome DevTools (клавиша F12) в меню **Settings** (горячая клавиша F1) → **Preferences** → **Sources** → **Search in anonymous and content scripts**.



Включение режима debug

По умолчанию на клиент загружаются минифицированные трудночитаемые файлы пакетов, библиотек и супербандлов, код которых отлаживать трудно и неудобно. Прежде чем приступить к отладке шаблона необходимо разминифицировать один или несколько требуемых модулей, в которых располагаются нужные шаблоны.

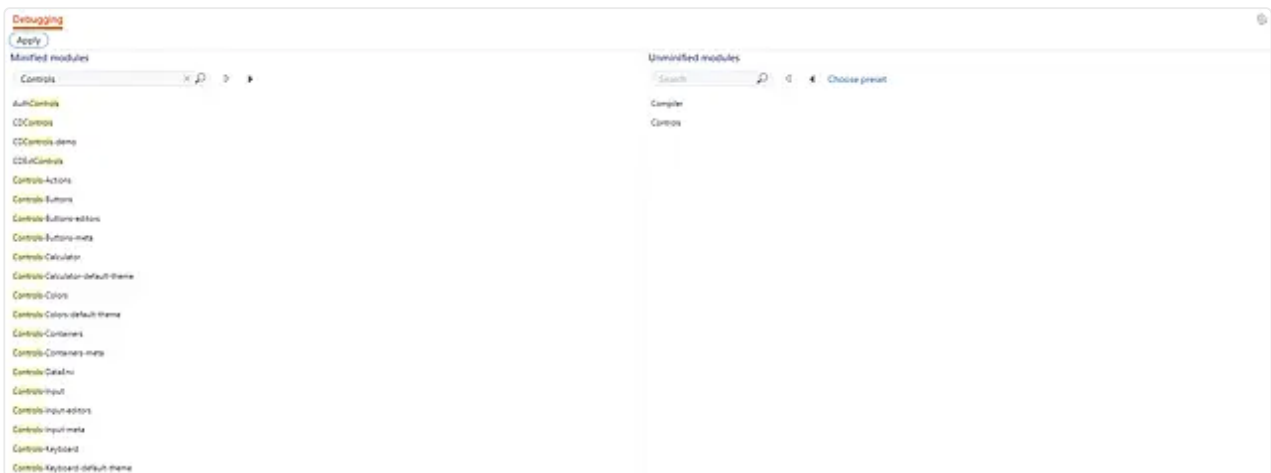
Модули, для которых нужно выполнять загрузку неминифицированных файлов, можно указать двумя способами:

1. Через консоль, установив значение для cookie s3debug:

▼ Пример кода

```
document.cookie="s3debug=Compiler,Module";
```

2. Через плагин [Wasaby Developer Tools](#), отметив в нем требуемые модули для разминификации.



Подмена шаблонов

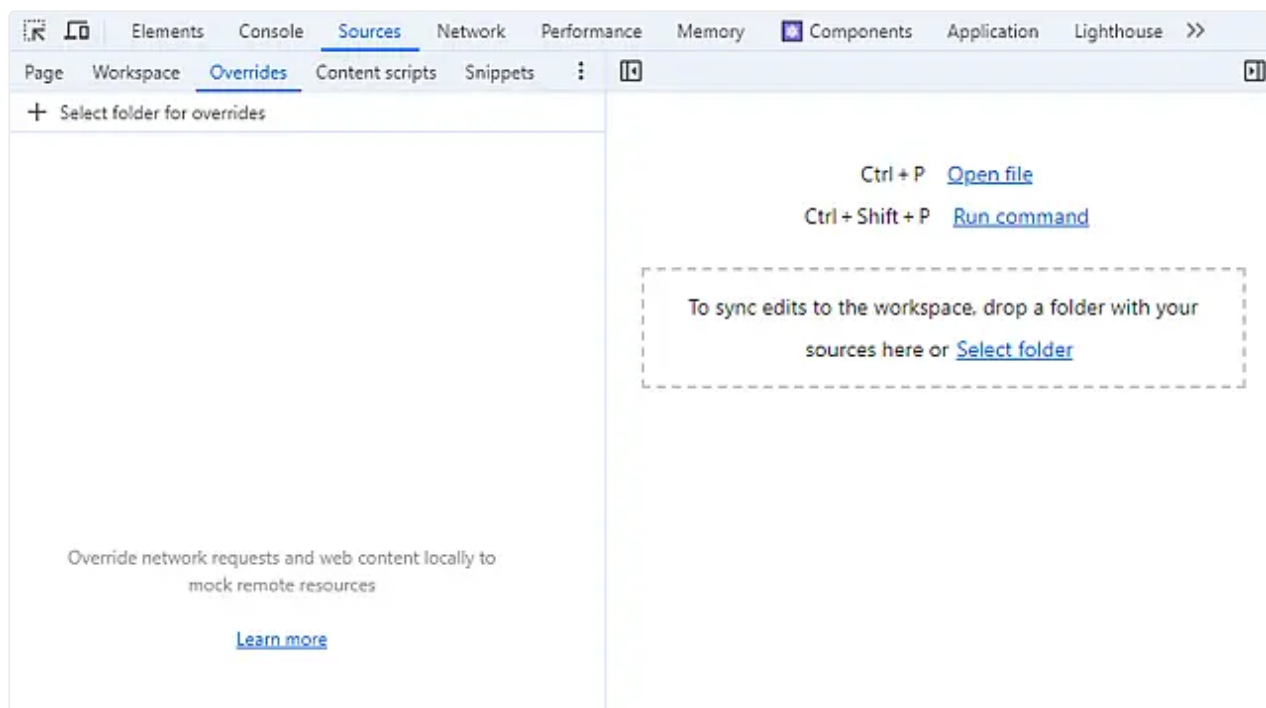
В некоторых случаях необходимо подменять исходный код шаблонов на клиенте. Это можно сделать двумя способами:

- С помощью [Fiddler](#);
- С помощью [Overrides](#) в Chrome DevTools.

Overrides в Chrome DevTools

Алгоритм включения Overrides:

1. Перейдите на страницу инструментов разработчика Chrome DevTools (клавиша F12);
2. Перейдите на вкладку **Sources** → подраздел **Overrides**;
3. Кликом по кнопке "+ Select folder for overrides" выберите директорию на диске, куда будут сохраняться подмененные файлы.

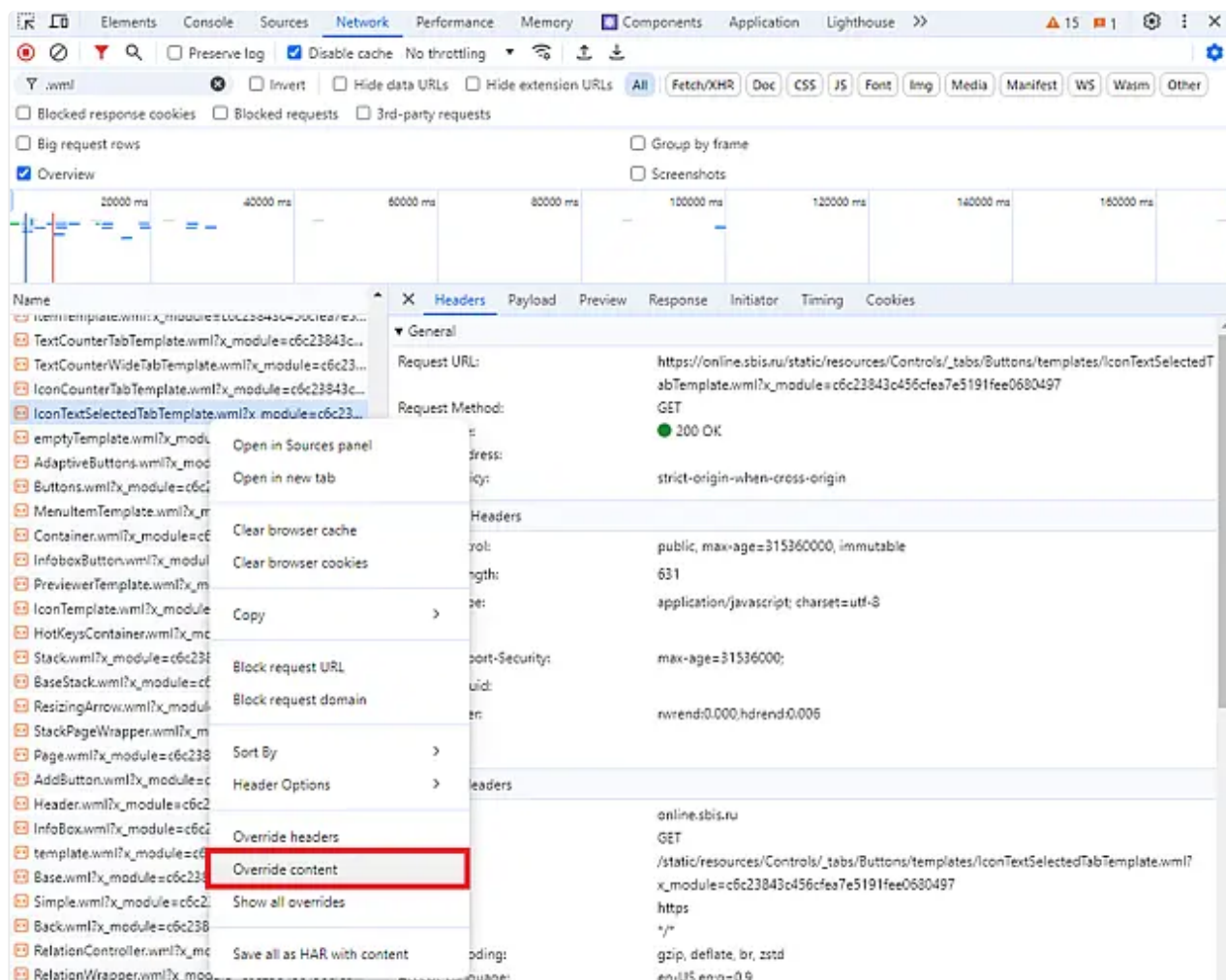


4. Разрешите доступ к указанной директории;

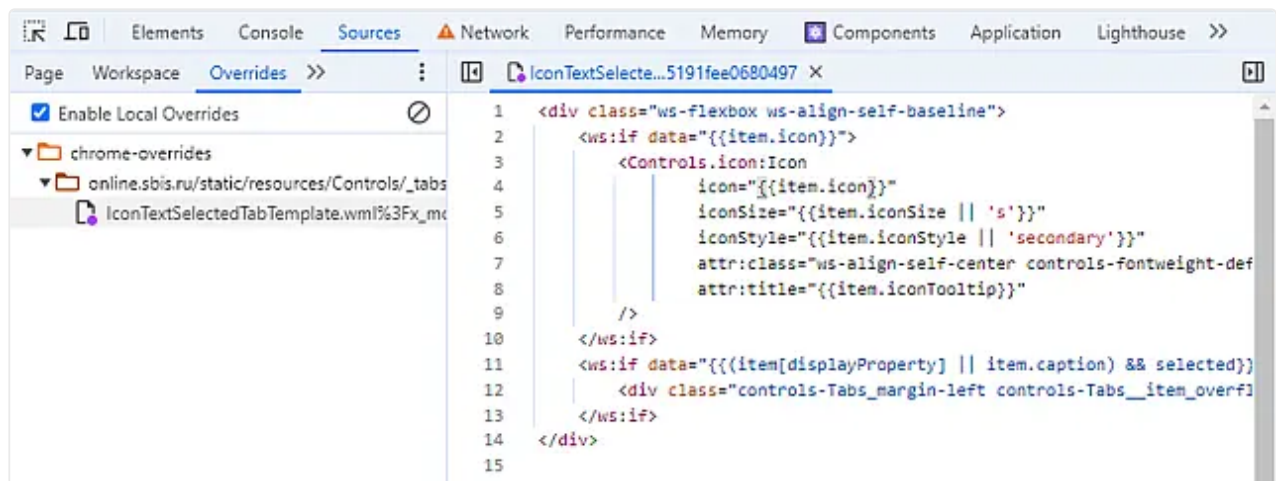
5. Отметьте флаг "Enable Local Overrides".

Для подмены содержимого файла выполните следующие шаги:

1. На вкладке **Network** выберите нужный файл, в контекстном меню выберите пункт "Override content";



2. После этого файл будет открыт на редактирование на вкладке **Sources** → подраздел **Overrides**:

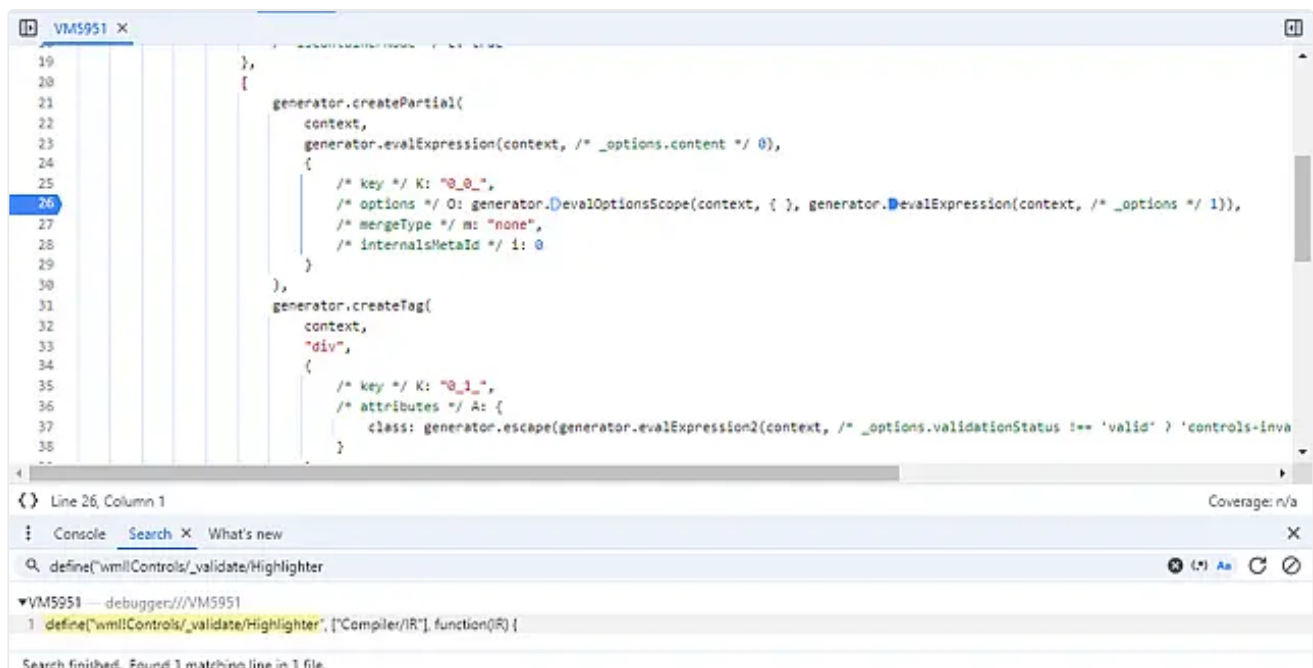


3. При перезагрузке страницы браузер будет использовать содержимое перезаписанных файлов.

Отладка шаблонов

Поскольку скомпилированный шаблон является JavaScript-модулем, то внутри него допустимо устанавливать точки останова.

Требуемый шаблон для отладки сперва необходимо найти. Для этого в поиске, который можно открыть комбинацией клавиш Ctrl + Shift + F, набрать `define("wml!<имя модуля>,"`, выбрать нужный шаблон и установить точки останова в нужных местах, например:



Метод debug()

Подобно тому, как в JavaScript есть выражение `debugger`, так и в шаблонах есть метод `debug()`, предназначенный для остановки выполнения построения шаблона в заданной точке.

В тексте исходного шаблона в требуемом месте необходимо разместить отладочное mustache-выражение в любом допустимом месте: в атрибутах и опциях HTML-тегов, шаблонов и контролов, а также в тексте.

▼ Пример кода

```

1 <Module.Control attr:class="{{ debug() }}" option="{{ debug() }}">
2   <div class="{{ debug() }}">
3     {{ debug() }}
4   </div>
5 </Module.Control>

```